



Licence Informatique – 1^{ère} Année
Rapport de SAE Algorithmique et Logique

projet:

General Problem Solver (GPS)

Auteurs :

medjoudj haithem :22514361
ferhaoui sofiane:22512969

Année universitaire 2024-2025

Introduction

Le contexte de ce projet est de refaire l'un des premiers programmes d'Intelligence Artificielle, le General Problem Solver (GPS) est pour objectif de résoudre des problèmes (faits initiaux et but) à l'aide des règles qui permettent de passer des faits initiaux aux buts.

I. Déroulement du Projet : Pour ce projet, nous avons travaillé en binôme en langage C, en découpant le projet en 4 parties. Nous avons testé et amélioré chaque partie avant de passer à la suivante.

*****Partie chargement des données** : Dans cette partie, nous avons appris à manipuler les fichiers en C et à lire dans un fichier à l'aide des fonctions `fopen`, `fgetc`, `fclose`. Nous avons utilisé la fonction `parseLine` pour découper les lignes en faits et repérer le nombre de faits. Nous avons utilisé des enregistrements et des tableaux comme structures de données permet de transformer un fichier texte en données.

*****Partie moteur de raisonnement basique** : Dans cette partie, nous avons implémenté un moteur basique qui part des faits initiaux vers le but. Nous avons utilisé les fonctions :
__ `applicable` pour vérifier si on peut appliquer la règle,
__ `appliquer` pour ajouter et supprimer des faits,
__ `but_atteint` pour vérifier si on est dans l'état final.
__ `trouver_regle` pour trouver une règle applicable
Ce moteur résout le problème des singes mais pas les autres, car il réapplique les règles en boucle. C'est pour cela que nous avons utilisé le `backtrack` pour mémoriser les états et pouvoir revenir en arrière.

*****Partie moteur amélioré** : Pour résoudre les trois problèmes, nous avons modifié la fonction `solve_prblm` et utilisé le `backtracking`. Il permet de mémoriser les états dans un enregistrement `etat` et de revenir en arrière sans entrer dans une boucle infinie comme avec le moteur basique.

au début, on a utilisé une variable booléenne appelée `used` comme un champ dans la structure `action` pour dire si une règle était déjà appliquée, ça a permis de résoudre le `problem school` (comme mentionné dans le pdf de la partie 3) mais le champ `used` n'a pas permis de résoudre le `problem de blocs`.

L'ensemble de la logique du `backtracking` est regroupée dans la fonction `solve_prblm` qui cherche une règle applicable et l'applique, mémorise l'état, et revient en arrière [comme dans le cas `blocs`]

dans notre implémentation, le `backtracking` est géré que par la variable `IR` (on a supprimé le champ de `"used"` dans `action`) `IR` permet de rechercher des règles à partir d'une position dans le tableau.

*chaque état est stocké dans `tab_etat`

*`IR` permet de reprendre la recherche à partir d'un indice

*pour rattraper l'absence du champ `used` on a implémenté `etat_deja_vu` qui vérifie si l'état était déjà fait par le moteur (comme expliqué en dessous).

lors des tests sur le problème des blocs et `school`, le moteur génère des boucles infinies à

l'affichage, le backtrack ne suffisait pas il revisite un état déjà exploré, ce qui fait une répétition . donc, nous avons ajouté `etat_deja_vu`, qui compare chaque nouvel état avec tous les états dans `tab_etat`. Si l'état a déjà été visité on cherche une autre règle applicable, ce qui évite les cycles et fait un affichage correct.

*****Partie formalisation du problème loup, chèvre, berger :** Nous avons réussi à formaliser le problème dans un fichier `chou.txt`. Nous avons défini les préconditions de manière à éviter les cas non permis

(loup,chèvre seuls)

et (chèvre,chou seuls).

Le GPS a réussi à atteindre le but après 5 états. (état 0 comme état initial)

remarque:

on précise que chaque état est affiché avec les faits courants pendant cet état suivi par la règle appliqué pour avoir ces faits.

II. Résultats du projet

II.1 Le programme

Nous avons utilisé des structures de données :

l'enregistrement action : contient les champs nom de l'action, les préconditions et les faits à modifier pour chaque règle (add, delete).

l'enregistrement etat : mémoriser les faits et des fonctions pour organiser notre code comme enregistrer qui sert à charger les données du fichier

Nous avons bien structuré et identifié notre code en utilisant des noms de fonctions clairs et des commentaires.

Il y a également des petits messages de "debug" pour vérifier le fonctionnement du backtracking ("IR mis a jour a", "cycle detecte"....)

II.2 Les fonctionnalités

Le programme permet de résoudre les trois problèmes (le singe, l'enfant à l'école et les blocs).

Il réussit à passer des faits initiaux au but.

Il permet aussi de résoudre le problème que nous avons formalisé(partie 4).

On peut donc généraliser ce solveur à n'importe quel problème qui respecte un certain format.

Nous avons ajouté un menu pour choisir quelle partie du projet exécuter et cela facilite les tests sur les 4 parties ce qui rend notre programme structuré, la partie 2 et 3 sont inclus dans une option parce que partie 3 est une version amélioré de la partie 2.

Nous n'avons pas traité les parties additionnelles.

Conclusion

Nous avons rencontré plusieurs difficultés au cours du développement. D'une part, des problèmes liés au backtracking et aux boucles infinies, que nous avons pu résoudre. D'autre part, dans la fonction supprimer, il a fallu gérer correctement le décalage des éléments après une suppression, ainsi que le caractère nul de fin de chaîne ('\0'). Enfin, lors de la formalisation du problème en partie 4, nous avons été confrontés à un problème d'encodage du caractère è dans le mot chèvre, on a remplcé le è par un simple e.

Dans ce projet, nous avons appris :

manipulation des fichiers en C ;

structures de données ;

principe du backtracking dans un système de résolution

détection de cycles pour garantir la terminaison de recherche.